

Embedded System
Serial port communication with Blink LEDs
Project Report(Assignment)



Submitted by:

Huzaifa Shahbaz
21-CE-009

Aqsa Shah
21-CE-039

Submitted To:

Sir Raza Ali Shah
Department of Computer Engineering,
HITEC University, Taxila

Contents

CHAPTER#1 ABSTRACT	6
1.1ABSTRACT:.....	6
CHAPTER#2 INTRODUCTION	6
2.1INTRODUCTION:	6
CHAPTER#3 OBJECTIVES.....	7
3.1OBJECTIVES:.....	7
CHAPTER#4 EXPECTED OUTCOME	7
4.1EXPECTED OUTCOME:	7
CHAPTER#5 METHODOLOGY	8
5.1 METHODOLOGY:	8
CHAPTER#6 PROBLEM STATEMENT	8
6.1PROBLEM STATEMENT:.....	8
CHAPTER#7 COMPONENTS USED.....	8
7.1COMPONENTS USED:	8
CHAPTER#8 FUNCTIONAL REQUIREMENTS.....	9
8.1FUNCTIONAL REQUIREMENTS:.....	9
8.1.1SERIAL COMMUNICATION SETUP:	9
8.1.2COMPUTER RECEPTION:	9
8.1.3COMMAND DECODING:	9
8.1.4 LED CONTROL:.....	9
8.1.5 REAL-TIME OPERATION:	9
8.1.6 ERROR HANDLING:.....	10
8.1.7 SCALABILITY:	10
CHAPTER#9 NON-FUNCTIONAL REQUIREMENTS	10
9.1NON-FUNCTIONAL REQUIREMENTS:.....	10
9.1.1RELIABILITY:.....	10
9.1.2 LATENCY:	10
9.1.3 SCALABILITY:	10
9.1.4 USABILITY:.....	11
9.1.5 PORTABILITY:	11
9.1.6MAINTAINABILITY:	11

9.1.7POWER EFFICIENCY:	11
CHAPTER#10 SERIAL COMMUNICATION PORT	11
10.1SERIAL COMMUNICATION PORT:	11
10.1.1TRANSMISSION OF DATA:	11
10.1.2 BAUD RATE:	12
10.1.3PROTOCOL CONFIGURATION:	12
10.1.4 PHYSICAL INTERFACE:	12
10.1.5 SOFTWARE TOOLS:	12
10.1.6APPLICATIONS:	12
CHAPTER#11 CODES OF SERIAL PORT COMMUNICATION WITH BLINK LEDs	12
11.1 CODES OF SERIAL PORT COMMUNICATION WITH BLINK LEDs:	12
11.1.1 MATLAB CODE OF SERIAL PORT COMMUNICATION WITH BLINK LEDs:	13
11.1.2 ARDUINO CODE OF SERIAL PORT COMMUNICATION WITH BLINK LEDs:	14
CHAPTER#12 RESULTS	14
12.1 RESULTS:	14
12.1.1 RESULT OF MATLAB:	15
12.1.2 RESULT OF ARDUINO UNO:	15
CHAPTER#13 CONCLUSION	15
13.1 CONCLUSION:	15

TABLE OF FIGURES

Figure 12.1.1:RESULTS OF MATLAB OF SERIAL PORT COMMUNICATION WITH LEDs.....	15
Figure 12.1.2:RESULT OF ARDUINO UNO	15
Figure 12.1.3:FINAL RESULT OF SERIAL PORT COMMUNICATION WITH BLINK LEDs.....	15

CHAPTER#1 ABSTRACT

1.1ABSTRACT:

The "Serial Port Communication with Blink LEDs" project has designed a serial communication that can control the LED states through a microcontroller. This, in fact, connects a computer with a microcontroller so that commands will be sent to and processed by the microcontroller. The microcontroller has to decode these commands then toggle and blink LEDs in different patterns. Thus, this project combines embedded systems and serial protocols in real-time control applications.

CHAPTER#2 INTRODUCTION

2.1INTRODUCTION:

Serial communication is an important mode of data transfer between two devices. This is widely used in embedded systems for communication that is efficient and reliable. Hence, this project titled 'Serial Port Communication with Blink LEDs' deals with the serial communication protocols that help control the speed and state of LEDs only. Here, LED circuits are connected between the public computer and the microcontroller via a serial port and commands from the computer for on, off and blink states for model LEDs can be sent via a computer to the microcontroller. This brings together an interaction between hardware and software, showcasing how microcontrollers interpret incoming data through serial interfaces and perform their programmed actions. This is a pretty simple application, yet very effective, demonstrating some important elements of communication protocols and an embedded system control.

CHAPTER#3 OBJECTIVES

3.1OBJECTIVES:

1. Learn the concepts of serial communication and implement it into embedded systems.
2. Create a system for reliable communication between a computer and a microcontroller using a serial port connection.
3. Control the LED by decoding the serial commands sent from the computer to the microcontroller.
4. Execute different LED blinking patterns as a result of commands received from the microcontroller.
5. Exhibit the application of serial communication in practical hardware control in real time.

CHAPTER#4 EXPECTED OUTCOME

4.1EXPECTED OUTCOME:

- 1.A functional system enabling real-time communication between a computer and a microcontroller via a serial port.
- 2.Dynamic control of LEDs, including turning them on/off and executing custom blinking patterns, through serial commands.
- 3.A reliable protocol for decoding and executing serial commands in the microcontroller.
- 4.Enhanced understanding of serial communication principles and their practical applications in embedded systems.
- 5.A demonstration of the project's scalability for controlling other hardware components beyond LEDs.

CHAPTER#5 METHODOLOGY

5.1 METHODOLOGY:

This methodology develops a serial interface between a computer and an Arduino to define a protocol for controlling LED actions. It also teaches the microcontroller how to decode commands and uses terminal software for testing and debugging. Improvements are made by optimizations for data transfer through error handling and feedback mechanism, followed by demonstrating variable control over LEDs through real-time serial communication. This is ensured by integrating all components for full demonstration of dynamic control over LEDs through real-time serial communication.

CHAPTER#6 PROBLEM STATEMENT

6.1PROBLEM STATEMENT:

Usually, a device such as LEDs is operated by direct human interface or hardcoded logic, which makes it harder to operate or control dynamically. Therefore, easy, efficient mechanism required to control LEDs remotely and possibly run various patterns in real time. Serial communication is seen to provide such a way out by creating seamless communication between computer and microcontroller so that hardware control commands could be executed accordingly. This mobile project handles the major task of serial communication of LED control, which may have various means of software and embedded system integration.

CHAPTER#7 COMPONENTS USED

7.1COMPONENTS USED:

1: ARDUINO

CHAPTER#8 FUNCTIONAL REQUIREMENTS

8.1FUNCTIONAL REQUIREMENTS:

The functional requirements of Serial port communication with Blink LEDs are:

8.1.1SERIAL COMMUNICATION SETUP:

The system should develop and establish a reliable serial communication link between the computer and the microcontroller.

8.1.2COMPUTER RECEPTION:

The microcontroller should be capable of receiving serial commands that are being sent to it from the computer.

8.1.3COMMAND DECODING:

These serial commands must then be interpreted and processed within the system for the appropriate LED action to be identified.

8.1.4 LED CONTROL:

The microcontroller is supposed to perform actuation of LED's ON/OFF or blinking/turning OFF based on the decoded commands.

8.1.5 REAL-TIME OPERATION:

The system should respond in real-time to the commands received by a serial link and should have minimal latency.

8.1.6 ERROR HANDLING:

The system should incorporate the facilities to take care of either invalid or corrupted commands without attributing it to any dislocation in functionality.

8.1.7 SCALABILITY:

The design must enable easy extension and incorporation of additional control for other LEDs or hardware in the future.

CHAPTER#9 NON-FUNCTIONAL REQUIREMENTS

9.1NON-FUNCTIONAL REQUIREMENTS:

The non-functional requirements of Serial port communication with Blink LEDs are:

9.1.1RELIABILITY:

There should be a complete contact between a computer and microcontroller with no data loss.

9.1.2 LATENCY:

The response time for executing commands should be minimal to support real-time operation.

9.1.3 SCALABILITY:

It should add LED or device without making much modification in the system.

9.1.4 USABILITY:

It should be comfortable and user friendly to send serial commands through any terminal like software or things alike.

9.1.5 PORTABILITY:

These designs work with several microsystems and operating systems through some modifications.

9.1.6 MAINTAINABILITY:

Documentation and modularization should be done in the codebase to allow easy debugging and further enhancement in the future.

9.1.7 POWER EFFICIENCY:

Power consumption during active and idle conditions of microcontroller and LEDs must be reduced by the system.

CHAPTER#10 SERIAL COMMUNICATION PORT

10.1 SERIAL COMMUNICATION PORT:

A serial communication port is the part where interfacing involves transferring data or communicating between two distinct devices via a specific serial interface, such as UART (Universal Asynchronous Receiver/Transmitter). In this project, it is to be used by a computer and a microcontroller to establish connection for common purposes. Among the key features and components of a serial communication port include:

10.1.1 TRANSMISSION OF DATA:

Bitwise data communication ensures the simplicity, streamlined process, and a low-cost implementation.

10.1.2 BAUD RATE:

Success in communication between two devices may only be achieved when their baud rates are configured to be the same for setting data transmission.

10.1.3 PROTOCOL CONFIGURATION:

Common parameters involve data bits (8 bits, for example), parity (for example, none), and stop bits (e.g., 1).

10.1.4 PHYSICAL INTERFACE:

One of the most common interfaces is typically UART pins on a microcontroller or USB-to-serial converters for connecting to a computer.

10.1.5 SOFTWARE TOOLS:

A serial terminal program like PuTTY, Tera Term, or any app of your choice can be used to do the send-receive work.

10.1.6 APPLICATIONS:

Through serial port in project computer sends command to microcontroller for real time control of LED.

CHAPTER#11 CODES OF SERIAL PORT COMMUNICATION WITH BLINK LEDs

11.1 CODES OF SERIAL PORT COMMUNICATION WITH BLINK LEDs:

In this assignment we are using MATLAB and Arduino for the serial port communication with blink LEDs.

11.1.1 MATLAB CODE OF SERIAL PORT COMMUNICATION WITH BLINK LEDs:

```
% Define the serial port and baud rate
serialPort = 'COM3'; % Replace with your Arduino's COM port
baudRate = 9600;
% Create the serial object
arduinoSerial = serial(serialPort, 'BaudRate', baudRate);
% Open the serial port
fopen(arduinoSerial);
disp('Serial port opened.');
```

% Send commands to blink LED

```
pause(2); % Wait for Arduino to initialize
fprintf(arduinoSerial, '1'); % Turn LED ON
disp('LED ON');
```

% Wait 1 second

```
pause(1);
fprintf(arduinoSerial, '0'); % Turn LED OFF
disp('LED OFF');
```

% Close the serial port

```
fclose(arduinoSerial);
delete(arduinoSerial);
disp('Serial port closed.');
```

11.1.2 ARDUINO CODE OF SERIAL PORT COMMUNICATION WITH BLINK LEDs:

```
void setup() {  
    pinMode(13, OUTPUT); // LED pin (onboard LED for Arduino Uno)  
    Serial.begin(9600); // Initialize serial communication at 9600 baud  
}  
  
void loop() {  
    if (Serial.available() > 0) {  
        char command = Serial.read(); // Read the command  
        if (command == '1') {  
            digitalWrite(13, HIGH); // Turn LED ON  
        } else if (command == '0') {  
            digitalWrite(13, LOW); // Turn LED OFF  
        }  
    }  
}
```

CHAPTER#12 RESULTS

12.1 RESULTS:

The results of MATLAB and Arduino uno of serial port communication blink with LEDs are:

12.1.1 RESULT OF MATLAB:

The result of MATLAB of serial port communication blink with LEDs as shown below:

Name ▲	Value
ans	1x2 string
arduinoSerial	1x1 Serialport
baudRate	9600
serialPort	'COM4'

Figure 12.1.1:RESULTS OF MATLAB OF SERIAL PORT COMMUNICATION WITH LEDs

12.1.2 RESULT OF ARDUINO UNO:

The result of ARDUINO UNO of serial port communication blink with LEDs as shown below:

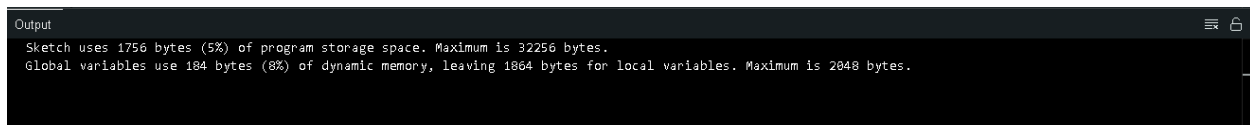


Figure 12.1.2:RESULT OF ARDUINO UNO



Figure 12.1.3:FINAL RESULT OF SERIAL PORT COMMUNICATION WITH BLINK LEDs

CHAPTER#13 CONCLUSION

13.1 CONCLUSION:

In conclusion, the Serial Communication Port has successfully demonstrated controlling indicators through the serial port. The communication serial interface is meant for applications involving

embedded systems. This project provides an efficient interface between the computer and the microcontroller for real-time control of hardware components. The commands sent via the serial port are interpreted and executed. The flexibility and practicality of serial communication for dynamic applications can be demonstrated by this application. It would improve the knowledge of data transmission protocols and form the base for future extensions of IoT and automation projects. Therefore, this project revives the need for the serial communication as a basic tool in present-day embedded system design.

